实验 2　openEuler Shell 编程基础入门

一、实验目标

(1)了解 openEuler 系统的 Shell 工作原理

(2)掌握 openEuler 系统的 Shell 基础编程

二、背景知识

Shell 是一个用 C 语言编写的程序，它是用户使用 openEuler 的桥梁。Shell 既是一种命令语言，又是一种程序设计语言。Shell 是指一种应用程序，这个应用程序提供了一个界面，用户通过这个界面访问操作系统内核的服务。Ken Thompson 的 sh 是第一种 Unix Shell，Windows Explorer 是一个典型的图形界面 Shell。

openEuler 系统的 Shell 相当于操作系统的“一层外壳”，它是命令语言解释器，它为用户提供了使用操作系统的接口，它不属于内核，而是在内核之外以用户态方式运行。它的基本功能是解释并执行用户打入的各种命令，实现用户与 openEuler 内核的接口。

在启动 openEuler 系统后，内核会为每个终端用户建立一个进程去执行 Shell 解释程序，它的执行过程遵循以下步骤：

(1)读取用户由键盘输入的命令；

(2)对命令进行分析，以命令名为文件名，并将其他参数改造为系统调用 execve()参数处理所要求的格式；

(3)终端进程(shell)调用 fork()或者 vfork()建立一个子进程；

(4)子进程根据文件名（命令名）到目录中查找有关文件，将他调入内存，并创建新的文本段，并根据写时拷贝的方式创建相应的数据段、堆栈段；

(5)当子进程完成处理或者出现异常后，通过 exit()或_exit()函数向父进程报告；

(6)终端进程调用 wait 函数来等待子进程完成，并对子进程进行回收；

常见的 Shell 有：Bourne shell（sh）、Korn shell（ksh）、Bourne Again shell（bash）、POSIX shell（sh）、C shell（csh）、TENEX/TOPS C shell（tcsh）。

接下来以命令 ls 为例解释一下 Shell 命令的运行流程。

命令 ls 的执行过程如下：

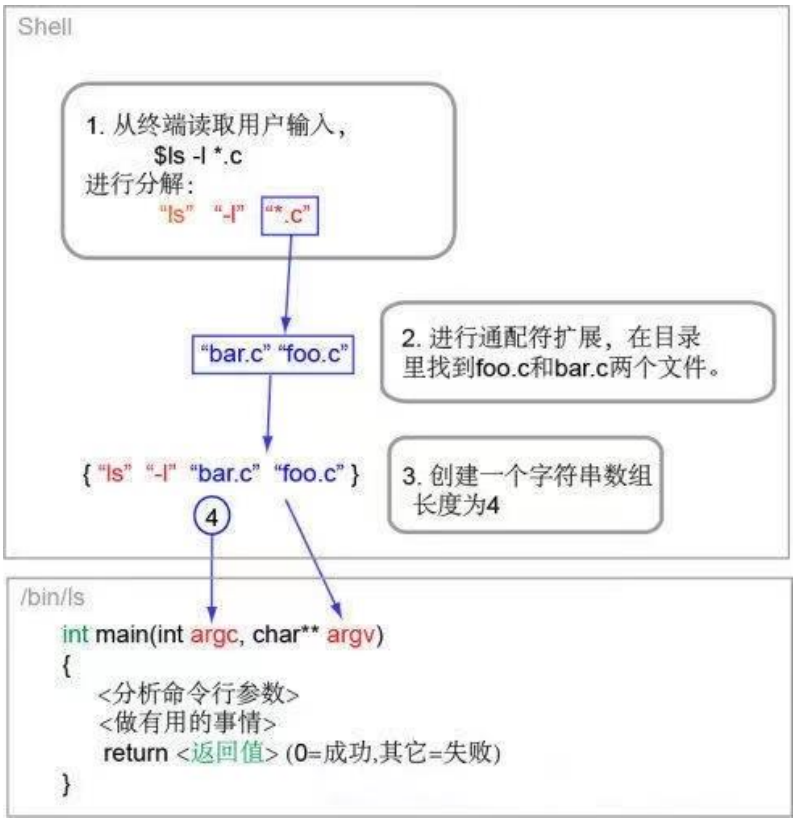


图 2.1　openEuler 中“ls”命令解析过程

(1)Shell 不断询问是否有键盘输入，用户以 enter 结束键盘输入后，Shell 程序以空格为分隔符，得到"ls"，"-l"，"*.c"，三个字符串。

(2)Shell 发现第三个字符串包含"*"通配符，就会在当前路径(或指定路径)下搜索满足这个通配符的文件。有 bar.c 和 foo.c 两个文件，然后展开成{"ls"，"-l"，"bar.c"，"foo.c"}四个字符串，并保存在 argv 这个二维字符数组中，并赋值 argc 为 4。

(3)然后 Shell 搜索 PATH 这个全局变量保存的路径，寻找一个叫"ls"的命令。如果存在，则 fork()子进程调用 exec()函数加载运行 ls 命令。ls 的 main(int argc, string ** argv)函数的参数就是从 Shell 那里得来的。若不存在就会提示用户"command not found"。

(4)ls 程序执行后，Shell 会分析 argc 和 argv，"-l"表示这是一个选项开关，表明用户想得到文件的详细信息，并把这个开关量保存，以便最后输出的时候选择信息的格式。

(5)ls 执行完结束后，退出并返回退出码，若成功则返回 0，否则根据错误的不同返回不同的非零数字。

(6)Shell 在得到成功退出码后，继续等待用户输入。

Shell 的另一个重要特性是它自身就是一个解释型的程序设计语言，Shell 程序设计语言支持绝大多数在高级语言中能见到的程序元素，如函数、变量、数组和程序控制结构。Shell 编

程语言简单易学，任何在提示符中能键入的命令都能放到一个可执行的 Shell 程序中。 当普通用户成功登录，系统将执行一个称为 Shell 的程序，并出现提示符。 “$”作为普通用户的提示符， “#”作为超级用户的提示符。一旦出现了 Shell 提示符，就可以键入命令名称及命令所需要的参数。Shell 将执行这些命令。如果一条命令花费了很长的时间来运行，或者在屏幕上产生了大量的输出，可以从键盘上按 ctrl+c 发出中断信号来中断它。当用户准备结束登录对话进程时，可以键入 logout 命令、exit 命令或文件结束符（EOF）（按 ctrl+d 实现），结束登录。

三、实验步骤

3.1 openEuler Shell 编程案例

(1)案例 1：编写 Shell 脚本，创建文件名为“firstShell.sh”的文件，在 Shell 脚本中实现变量的加减乘除比较操作并输出计算结果。

实验步骤如下：

①在命令行中执行命令 touch firstShell.sh 以创建第一个 Shell 脚本。

```
[root@localhost ~]# touch firstShell.sh
```

②在命令行中执行命令 vi firstShell.sh 进入 Shell 脚本开始编写。

```
[root@localhost ~]#vi firstShell.sh
```

③点击键盘上“i”按键开始写入，写入脚本如下：

```
#! /bin/sh
a=10
b=20
echo “a=$a”
echo “b=$b”
echo ‘a+b=’ `expr $a + $b`
echo ‘a-b=’`expr $a - $b`
echo ‘a*b=’ `expr $a \* $b`
echo ‘a/b=’ `expr $a / $b`
if [ $a==$b ]
then
    echo ‘a=b’
else
    echo ‘a<>b’
fi
```

④点击键盘上“Esc”键，输入“:wq”并回车，退出 vim。

⑤输入命令 sh firstShell.sh 或 bash firstShell.sh 运行 firstShell.sh 脚本，最后查看输出结果。

```
[root@localhost ~]# sh firstShell.sh.sh
```

```
a=10
b=20
a+b= 30
a-b= -10
a*b= 200
a/b= 0
a<>b
```

(2)案例 2：编写 Shell 脚本，完成一个简单的 while 循环实现 1 到 10 的输出

实验步骤如下：

①在命令行中执行命令 touch secondShell.sh 以创建第二个 Shell 脚本。

```
[root@localhost ~]# touch secondShell.sh
```

②在命令行中执行命令 vi secondShell.sh 进入 Shell 脚本开始编写。

```
[root@localhost ~]# vi secondShell.sh
```

③点击键盘上“i”按键开始写入，写入脚本如下：

```
#! /bin/sh
i=1
while [ $i -le 10 ]
do
echo $i
let i++
done
```

④点击键盘上“Esc”键，输入“:wq”并回车，退出 vim。

⑤输入命令 sh secondShell.sh 或 bash secondShell.sh 运行 secondShell.sh 脚本，最后查看输出结果。

```
[root@localhost ~]# sh secondShell.sh
1
2
3
4
5
6
7
8
9
10
```

(3)案例 3：编写 Shell 脚本，完成一个对端口 eth33 的实时流量监控并输出

实验步骤如下：

①在命令行中执行命令 touch thirdShell.sh 以创建第三个 Shell 脚本。

```
[root@localhost ~]# touch thirdShell.sh
```

②在命令行中执行命令 vi thirdShell.sh 进入 Shell 脚本开始编写。

```
[root@localhost ~]# vi thirdShell.sh
```

③点击键盘上“i”按键开始写入，写入脚本如下：

```
#!/bin/bash
eth0=$1
echo  -e  "in--out  "
while true; do
    old_in=$(cat /proc/net/dev |grep $eth0 |awk '{print $2}')
    old_out=$(cat /proc/net/dev |grep $eth0 |awk '{print $10}')
    sleep 1
    new_in=$(cat /proc/net/dev |grep $eth0 |awk '{print $2}')
    new_out=$(cat /proc/net/dev |grep $eth0 |awk '{print $10}')
    in=$(printf "%.1f%s" "$(($new_in-$old_in)/1024))" "KB/s")
    out=$(printf "%.1f%s" "$(($new_out-$old_out)/1024))" "KB/s")
    echo "$in $out"
done
```

④点击键盘上“Esc”键，输入“:wq”并回车，退出 vim。

⑤输入命令 sh thirdShell.sh ens33 或 bash thirdShell.sh ens33 运行 thirdShell.sh 脚本，最后查看输出结果。

```
[root@localhost ~]# sh thirdShell.sh ens33
In—out
2.0KB/s 10.0KB/s
0.0KB/s 2.0KB/s
1.0 KB/s 3.0KB/s
2.0 KB/s 12.0KB/s
0.0 KB/s 2.0 KB/s
1.0 KB/s 5.0 KB/s
2.0 KB/s 9.0 KB/s
0.0 KB/s 2.0 KB/s
1.0 KB/s 6.0 KB/s
```

(4)案例 4：编写 Shell 脚本，要求输出 CPU、IO 接口、硬盘、内存的使用情况以及 TCP 连接的情况。

①在命令行中执行命令 touch forthShell.sh 以创建第四个 Shell 脚本。

```
[root@localhost ~]# touch forthShell.sh
```

②在命令行中执行命令 vi forthShell.sh 进入 Shell 脚本开始编写。

```
[root@localhost ~]# vi forthShell.sh
```

③点击键盘上“i”按键开始写入，写入脚本如下：

```
#!/bin/bash
function cpu(){

    util=$(vmstat | awk '{if(NR==3)print $13+$14}')
    iowait=$(vmstat | awk '{if(NR==3)print $16}')
    echo " CPU - Usage：${util}% , disk IO to be utilized accordingly：${iowait}:${iowait}%"

}
function memory (){
    total=`free -m |awk '{if(NR==2)printf "%.1f",$2/1024}'`
    used=`free -m |awk '{if(NR==2) printf "%.1f",($2-$NF)/1024}'`
    available=`free -m |awk '{if(NR==2) printf "%.1f",$NF/1024}'`
    echo " Memory - Total size: ${total}G , used: ${used}G , remainder: ${available}G"
}
disk(){
    fs=$(df -h |awk '/^\/dev/{print $1}')
    for p in $fs; do
        mounted=$(df -h |awk '$1=="'$p'"{print $NF}')
        size=$(df -h |awk '$1=="'$p'"{print $2}')
        used=$(df -h |awk '$1=="'$p'"{print $3}')
        used_percent=$(df -h |awk '$1=="'$p'"{print $5}')
        echo " Hard Disk - Mount Point: $mounted , Total size: $size , used: $used , Usage: $used_percent"
    done

}
function tcp_status() {
    summary=$(ss -antp |awk '{status[$1]++}END{for(i in status) printf i":"status[i]" "}')
    echo " TCP connection status - $summary"
}
cpu
memory
disk
tcp_status
```

④点击键盘上“Esc”键，输入“:wq”并回车，退出 vim。

⑤输入命令 sh forthShell.sh 或 bash forthShell.sh 运行 forthShell.sh 脚本，最后查看输出结果。

```
[root@localhost ~]# sh forthShell.sh
CPU-Usage:0%,disk IO to be utilized accordingly:0.0%
Memory – Toyal size:7.3G,used:0.7G,remainder:6.6G
Hard Disk - Mount Point:/,Total size:17G.used:8.0G,Usage:51%
Hard Disk – Mount Point:/boot,Total size:976M,,used:126M,Usage:14%
```

TCP connection status – LISTEN:2 ESTAB:2 State:1

四、实验讨论

(1)案例一中，我们实现了简单的算术运算，那么如何使用 Shell 脚本实现更复杂的数学计算，例如三角函数或指数函数？

(2)案例二中，我们使用 while 循环打印数字，那么如何使用 Shell 脚本创建更复杂的循环，例如嵌套循环或 do-while 循环？

(3)案例三中，我们实现了对一个端口流量的检测，那么如何编写一个 Shell 脚本，以监视多个端口或所有打开的端口的流量？

(4)案例四中，我们列出了系统资源的使用情况，那么如何使用 Shell 脚本监视特定进程的资源使用情况？